SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1005	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.

- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.

Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

ARCHITECTURE DESIGN ANALYSIS REPORT

Course: CSA10 / CSA1005 - Software Engineering | **CO2 Assessment Tool 2**

Project: Smart School Bus Tracking and Student Safety System

Student: M. Rudresh | **Register No.:** 192372339

1. Analyze System Requirements

System objectives: Provide safe and transparent student transportation by tracking buses in real time, maintaining student, bus and driver records, managing routes, logging boarding and drop-off, and alerting parents and school administrators about delays, deviations and incidents.

Functional requirements: (1) Secure login for parents, drivers and administrators; (2) create and update student, bus and driver profiles; (3) assign routes, stops and students to buses; (4) capture and display GPS location and estimated arrival; (5) record boarding/drop-off with timestamps; (6) detect route deviations and unexpected delays; (7) send alerts; (8) log incidents; (9) generate transportation and attendance reports.

Non-functional requirements: Responsive tracking updates, privacy of student data, high uptime during school travel hours, recovery from network failures, ability to add more buses and schools, and maintainable modular code.

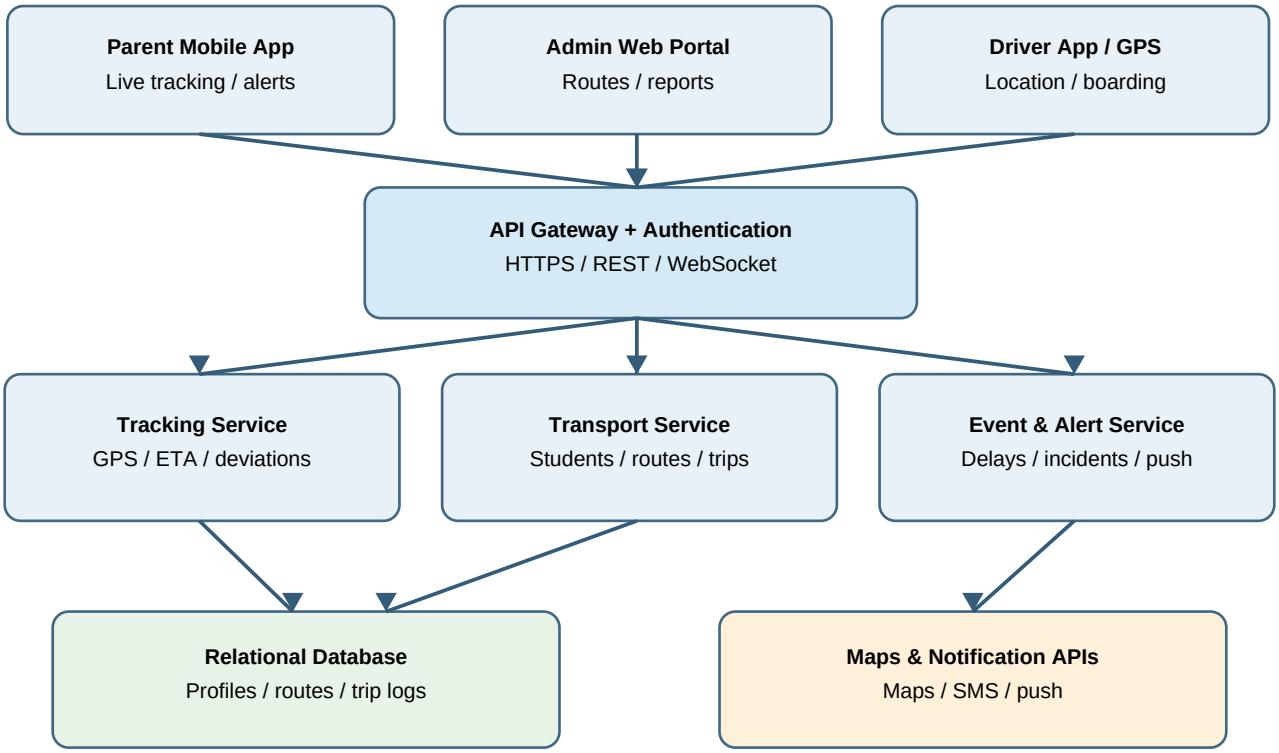
Key quality attributes: Scalability, maintainability, reliability, availability, performance and security influence the architecture. GPS data must be processed efficiently while personal records must remain access-controlled.

2. Select a Suitable Software Architecture

Selected style: Hybrid Client-Server and Layered Architecture with Event-Driven Notifications. Mobile/web clients connect to a central backend API. The backend separates presentation, business logic and data access; location updates and alerts are processed as events.

Justification: Client-server access provides one authoritative source of route and student information. Layers make individual features easier to develop, test and maintain. Event-driven alert handling prevents notification delivery from slowing GPS ingestion and makes future integrations easier.

3. Software Architecture Diagram



Communication: Clients use HTTPS REST APIs for records and commands; WebSocket or periodic HTTPS polling delivers location updates. The GPS source sends coordinates to the tracking service. Alert events trigger push/SMS services. Persistent data is stored in a relational database.

4. Explain Components and Their Interactions

Presentation clients: Parent app shows bus location, ETA and boarding alerts; driver app sends GPS and records attendance; admin portal manages drivers, buses, routes and reports.

API and authentication: Validates users and roles, accepts requests, and routes operations to appropriate services. **Tracking service:** Validates location updates, calculates ETA and checks route boundaries. **Transport service:** Manages master records and trip/boarding history. **Alert service:** Creates notifications for deviations, delays, incidents and boarding events. **Database:** Stores structured records and audit trails. **External APIs:** Supply maps and notification delivery.

Typical workflow: 1) Admin registers buses, drivers, students and routes. 2) Driver starts a trip and sends GPS updates. 3) Tracking service stores location and estimates arrival. 4) Parent views the bus on a map. 5) Driver records student boarding/drop-off. 6) Alert service notifies authorized parents. 7) Deviations and delays create alerts; administrators review incidents and reports.

5. Discuss Quality Attributes

Scalability: Stateless API instances can be replicated; GPS ingestion and notification workers can scale separately. **Security:** TLS, role-based access, secure authentication, minimal parent access to only their children, encryption and audit logging protect data. **Performance:** Cache current locations, index route/trip records and process alerts asynchronously.

Reliability: Retry failed GPS uploads and notification jobs; keep timestamped logs and backups. **Maintainability:** Clear layers, modular services, API contracts and automated tests reduce change impact. **Availability:** Health checks, redundant deployments, monitoring and database backups support service continuity; offline driver queues handle temporary connectivity loss.

6. Justify Architectural Decisions

Rationale: A layered client-server foundation is practical for a school transport application because it centralizes records while keeping user interfaces separate from core logic. Event-driven alerts handle unpredictable location and safety events without blocking primary operations.

Trade-offs: A central backend simplifies consistency but requires resilient hosting. Live updates improve visibility but use bandwidth and battery. Splitting tracking and alerts improves scaling but increases deployment complexity; for a small prototype they may begin as modules within one backend.

Future enhancements: Add RFID/QR boarding, geofencing, predictive ETA, multiple schools, analytics dashboards and emergency escalation through versioned APIs. Independent modules and documented interfaces support long-term maintenance and integration.

Conclusion: The proposed hybrid architecture provides a secure, scalable and maintainable basis for reliable real-time school bus tracking and student safety.